```
import streamlit as st

import mysql.connector

import pandas as pd

from datetime import datetime

from reportlab.pdfgen import canvas


# -----------------------------

# MYSQL CONNECTION

# -----------------------------

conn = mysql.connector.connect(

    host="localhost",

    user="root",

    password="Thulasi2005*",   # உங்கள் password

    database="pharmacy_db"

)

cursor = conn.cursor(dictionary=True)


# -----------------------------

# PAGE CONFIG

# -----------------------------

st.set_page_config(page_title="Pharmacy System", layout="wide")


# -----------------------------

# SESSION

# -----------------------------

if "logged_in" not in st.session_state:

    st.session_state.logged_in = False

if "role" not in st.session_state:
```

```python
    st.session_state.role = ""
if "cart" not in st.session_state:
    st.session_state.cart = []

# -----------------------------
# LOGIN
# -----------------------------
def login():
    st.title("🏥 Medical Shop Login")

    username = st.text_input("Username")
    password = st.text_input("Password", type="password")
    role = st.selectbox("Role", ["Admin", "Staff"])

    if st.button("Login"):
        cursor.execute(
            "SELECT * FROM users WHERE username=%s AND password=%s AND role=%s",
            (username, password, role)
        )
        user = cursor.fetchone()

        if user:
            st.session_state.logged_in = True
            st.session_state.role = role
            st.success("Login Successful")
            st.rerun()
        else:
            st.error("Invalid Credentials")
```

```
# -----------------------------
# MEDICINES
# -----------------------------
def medicines():
  st.title("💊 Medicines")

  name = st.text_input("Name")
  category = st.text_input("Category")
  price = st.number_input("Price", min_value=0.0)
  quantity = st.number_input("Quantity", min_value=0)
  expiry = st.date_input("Expiry")

  if st.button("Add Medicine"):
    cursor.execute(
      "INSERT INTO medicines(name,category,price,quantity,expiry_date) VALUES(%s,%s,%s,%s,%s)",
      (name, category, float(price), int(quantity), expiry)
    )
    conn.commit()

    cursor.execute(
      "INSERT INTO stock_log(medicine_id,change_qty,date) VALUES(LAST_INSERT_ID(), %s, %s)",
      (int(quantity), datetime.now())
    )
    conn.commit()
```

```python
        st.success("Medicine Added")

    df = pd.read_sql("SELECT * FROM medicines", conn)
    st.dataframe(df)

# ----------------------------
# STOCK
# ----------------------------
def stock():
    st.title("📦 Stock Management")

    df = pd.read_sql("SELECT * FROM medicines", conn)
    st.dataframe(df)

    med_id = st.number_input("Medicine ID", min_value=1)
    add_qty = st.number_input("Add Quantity", min_value=1)

    if st.button("Update Stock"):
        cursor.execute(
            "UPDATE medicines SET quantity = quantity + %s WHERE medicine_id=%s",
            (int(add_qty), int(med_id))
        )
        cursor.execute(
            "INSERT INTO stock_log(medicine_id,change_qty,date) VALUES(%s,%s,%s)",
            (int(med_id), int(add_qty), datetime.now())
        )
        conn.commit()
```

```
        st.success("Stock Updated")


# -----------------------------

# LOW STOCK

# -----------------------------

def low_stock():

    st.title("⚠️ Low Stock Alert")


    threshold = st.number_input("Threshold", value=10)


    df = pd.read_sql(

        "SELECT * FROM medicines WHERE quantity <= %s",

        conn,

        params=(threshold,)

    )


    st.dataframe(df)


# -----------------------------

# EXPIRY ALERT

# -----------------------------

def expiry_alert():

    st.title("⏳ Expiry Alert")


    today = datetime.now().date()


    df = pd.read_sql(

        "SELECT * FROM medicines WHERE expiry_date <= %s",
```

```
        conn,
        params=(today,)
    )

    st.dataframe(df)

# -----------------------------
# BILLING
# -----------------------------
def billing():
    st.title("🧾 Billing")

    meds = pd.read_sql("SELECT * FROM medicines", conn)

    if meds.empty:
        st.warning("No medicines available")
        return

    col1, col2, col3 = st.columns(3)

    with col1:
        selected = st.selectbox("Medicine", meds["name"])

    with col2:
        qty = st.number_input("Qty", min_value=1)

    with col3:
        if st.button("Add"):
```

```
        med = meds[meds["name"] == selected].iloc[0]

        if int(qty) > int(med["quantity"]):
            st.error("Not enough stock")
        else:
            st.session_state.cart.append({
                "id": int(med["medicine_id"]),
                "name": selected,
                "qty": int(qty),
                "price": float(med["price"]),
                "total": float(qty * med["price"])
            })

    st.subheader("Cart")

    if st.session_state.cart:
        df = pd.DataFrame(st.session_state.cart)
        st.dataframe(df)

        total = float(df["total"].sum())
        st.success(f"Total ₹{total}")

        if st.button("Generate Bill"):
            cursor.execute(
                "INSERT INTO sales(date,total_amount) VALUES(%s,%s)",
                (datetime.now(), float(total))
            )
            conn.commit()
```

```
        cursor.execute("SELECT LAST_INSERT_ID() AS id")
        sale_id = cursor.fetchone()["id"]

        for item in st.session_state.cart:
            cursor.execute(
                "INSERT INTO sale_items(sale_id,medicine_id,quantity,price)
VALUES(%s,%s,%s,%s)",
                (sale_id, item["id"], item["qty"], item["total"])
            )

            cursor.execute(
                "UPDATE medicines SET quantity = quantity - %s WHERE medicine_id=%s",
                (item["qty"], item["id"])
            )

        conn.commit()

        generate_pdf(sale_id, st.session_state.cart, total)

        st.success(f"Bill Generated ID: {sale_id}")
        st.session_state.cart = []

# -----------------------------
# PDF
# -----------------------------
def generate_pdf(sale_id, items, total):
    file = f"bill_{sale_id}.pdf"
```

```
    c = canvas.Canvas(file)

    y = 800
    c.drawString(200, y, "PHARMACY BILL")

    y -= 30
    c.drawString(100, y, f"Bill ID: {sale_id}")

    for item in items:
        y -= 20
        c.drawString(100, y, f"{item['name']} x{item['qty']} = Rs.{item['total']}")

    y -= 30
    c.drawString(100, y, f"Total: Rs.{total}")

    c.save()

# -----------------------------
# SALES REPORT
# -----------------------------
def sales_report():
    st.title("📊 Sales Report")

    df = pd.read_sql("SELECT * FROM sales", conn)
    st.dataframe(df)

    if not df.empty:
        st.success(f"Total Revenue ₹{df['total_amount'].sum()}")
```

```
# -----------------------------
# MAIN
# -----------------------------
if not st.session_state.logged_in:
    login()
else:
    st.sidebar.title("Pharmacy System")

    if st.sidebar.button("Logout"):
        st.session_state.logged_in = False
        st.rerun()

    if st.session_state.role == "Admin":
        menu = st.sidebar.selectbox("Menu", [
            "Medicines",
            "Billing",
            "Stock",
            "Low Stock",
            "Expiry Alert",
            "Sales Report"
        ])
    else:
        menu = st.sidebar.selectbox("Menu", ["Billing"])

    if menu == "Medicines":
        medicines()
    elif menu == "Billing":
```

```
    billing()
elif menu == "Stock":
    stock()
elif menu == "Low Stock":
    low_stock()
elif menu == "Expiry Alert":
    expiry_alert()
elif menu == "Sales Report":
    sales_report()
```